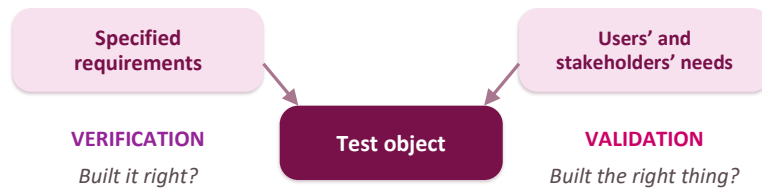


1. Fundamentals of Testing

A. What is testing? Test objectives



Testing is a set of activities to **find defects** and **evaluate the quality** of software work products. The work product being tested is the **test object**.

- **Verification**: checking the system meets its **specified requirements**
- **Validation**: checking it meets **users' and stakeholders' needs** in its real environment

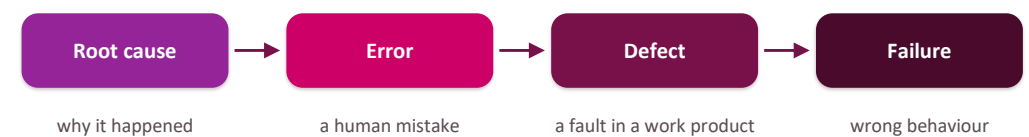
Dynamic testing runs the software; **static testing** (reviews and static analysis) examines it without running it.

Typical test objectives

- Evaluate work products such as requirements, user stories, designs and code
- Trigger failures and find defects
- Ensure the required coverage of the test object
- Reduce the risk of poor software quality
- Verify that specified requirements have been met
- Verify compliance with contracts, laws and regulations
- Give stakeholders the information they need to make decisions
- Build confidence in the quality of the test object
- Validate that the test object is complete and works as stakeholders expect

The objectives depend on the **context**: the work product, the test level, the risks, the SDLC and business factors.

B. Errors, defects, failures, root causes

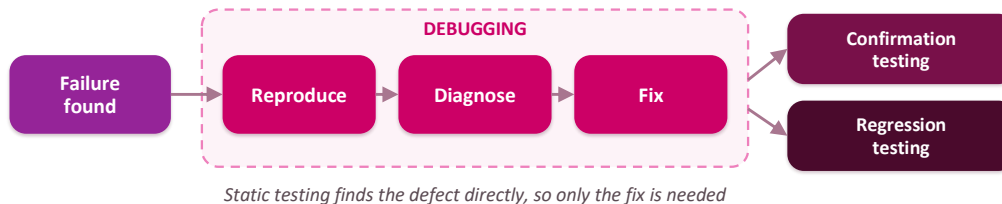


- People make **errors** (mistakes), e.g., because of time pressure, complexity, tiredness or lack of training
- An error can create a **defect** in documentation, code or a supporting work product such as a build file
- When defective code runs, it may cause a **failure**. Not every defect causes a failure, and failures can also come from **environmental conditions** such as radiation
- The **root cause** is the underlying reason the error happened. **Root cause analysis** tackles it, preventing similar defects in future

C. Seven testing principles

1. **Testing shows the presence, not the absence of defects**: testing can show defects exist, but can't prove there are none
2. **Exhaustive testing is impossible**: you can't test everything, so use test techniques, prioritisation and risk-based testing
3. **Early testing saves time and money**: defects removed early don't cause more defects later
4. **Defects cluster together**: a few components usually contain most of the defects (the Pareto principle)
5. **Tests wear out**: repeating the same tests finds fewer new defects, so update tests and data and write new ones
6. **Testing is context dependent**: no single approach suits every project
7. **Absence-of-defects fallacy**: a system with no known defects can still fail to meet users' needs

D. Testing vs debugging



Testing shows that something is wrong: dynamic testing triggers **failures**, static testing finds **defects** directly. **Debugging** is a separate activity: finding, analysing and removing the **cause** of a failure.

- **Confirmation testing** then checks the fix worked, ideally run by the person who ran the original test
- **Regression testing** checks the fix hasn't broken anything else

E. Why is testing necessary?

Testing is a form of **quality control** that helps a project meet its goals for scope, time, quality and budget. It contributes by:

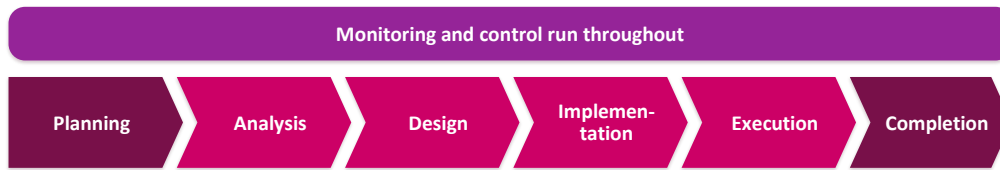
- Finding defects **cost-effectively** so they can be fixed, which improves quality indirectly
- **Evaluating quality directly** at each stage of the SDLC, to support decisions such as whether to release
- **Representing users** on the project when involving real users isn't practical
- Helping meet **contractual, legal and regulatory** requirements

F. Testing and quality assurance

Testing and quality assurance (QA) are related, but **not the same**:

- **Testing is product-oriented and corrective**. It is a form of **quality control**, and its results are used to fix defects
- **QA is process-oriented and preventive**. It sets up and improves processes, on the basis that a good process gives a good product. Test results give QA feedback on how well the processes work

G. Test activities and testware



Testing is made up of groups of activities. They are tailored to the context and often repeated; **monitoring and control** happen throughout.

- **Planning:** set the test objectives and choose the approach. *Produces:* test plan, schedule, risk register, entry and exit criteria
- **Monitoring and control:** compare progress with the plan and act to stay on track. *Produces:* progress reports, control directives, risk information
- **Analysis** (what to test): study the test basis to find and prioritise test conditions. *Produces:* test conditions (e.g., acceptance criteria), defect reports on the test basis
- **Design** (how to test): turn test conditions into test cases; identify coverage items, test data and environment needs. *Produces:* test cases, test charters, coverage items
- **Implementation:** build what's needed to run the tests: procedures, suites, scripts, test data, the execution schedule and the environment. *Produces:* test procedures, scripts, suites, test data, stubs, drivers, simulators
- **Execution:** run the tests, compare actual with expected results, log results and report anomalies. *Produces:* test logs, defect reports
- **Completion:** archive testware, shut down the environment, capture lessons learned and report. *Produces:* test completion report, improvement actions, change requests

H. Context and traceability



Two-way links show coverage, residual risk and the impact of changes

How testing is done depends on its **context**: stakeholders, the team, the business domain, technical factors, project constraints, the organisation, the SDLC and the tools available. These shape the strategy, techniques, automation, coverage, documentation and reporting.

Traceability links the test basis to test conditions, test cases, results and defects. It lets you measure coverage, see the impact of changes, support audits and IT governance, and report progress clearly.

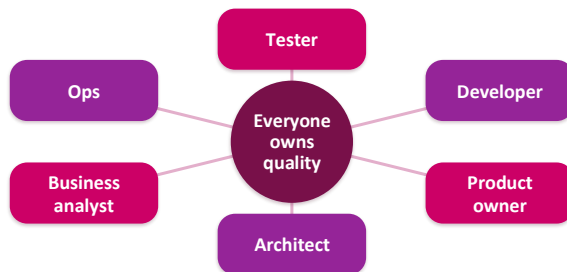
I. Roles and skills

- **Test management role:** responsible for the test process and test team; mainly planning, monitoring, control and completion
- **Testing role:** responsible for the technical side; mainly analysis, design, implementation and execution
- Different people can take these roles, and one person can hold both

Generic skills

Good testers need testing knowledge; thoroughness, curiosity and attention to detail; communication and teamwork; analytical, critical and creative thinking; and technical and domain knowledge. Testers often bring bad news, so report defects **constructively**: **confirmation bias** makes people resist information that contradicts what they believe.

J. Whole team approach



Comes from Extreme Programming (XP): anyone with the right skills can do any task, everyone is responsible for quality, and the team shares a workspace.

Benefits: better team dynamics, communication and collaboration, and everyone's skills used together. It is **not always suitable**: safety-critical systems, for example, need a high level of test independence.

K. Independence of testing



More independent testers find different defects, because they don't share the author's assumptions and biases. Familiarity still matters: developers can find many defects in their own code efficiently.

Benefits: they recognise different kinds of failures and challenge assumptions made during specification and coding.

Drawbacks: they can become isolated from the team, communication suffers, developers may feel less responsible for quality, and testers may be seen as a bottleneck.

L. Key terms

- **Test object:** the work product being tested
- **Test basis:** the information used to decide what to test (e.g., requirements)
- **Test condition:** something testable, identified from the test basis
- **Test case:** preconditions, inputs, actions, expected results and postconditions
- **Test procedure:** test cases in the order they will run, plus set-up and wrap-up steps
- **Test suite:** a set of test procedures or scripts run together
- **Testware:** all the work products created during testing
- **Coverage:** the percentage of coverage items the tests have exercised